(continued)

How does Elasticsearch deduce the data type when we provide the values as an array during dynamic mapping? The data type is derived from the type of the first element in the array. For example, `"John Smith"` is a string, so name is a `text` type despite its representation as an array.

An important point to consider when creating arrays is that we cannot mix different types in an array. For example, we cannot declare the name field like this: `"name": ["John Smith", 13, "Neverland"]`. This is illegal because the field consists of multiple types.

4.6.4 The flattened data type

So far, we've looked at indexing fields parsed from a JSON document. Each field is treated as individual and independent when it is analyzed and stored. However, sometimes we may not need to index all the subfields as individual fields, thus avoiding the expensive analysis process. Think of a stream of chat messages on a chat system, the running commentary during a live football match, or a doctor taking notes about a patient's ailments. We can load this kind of data as one big blob rather than declaring each field explicitly (or deriving it dynamically). Elasticsearch provides a special data type called `flattened` for this purpose.

A `flattened` field holds information in the form of one or more subfields, and each subfield's value is indexed as a keyword. None of the values are treated as `text` fields, and thus they do not undergo the text analysis process.

Consider an example of a doctor taking running notes about their patient during a consultation. The mapping consists of two fields: the patient's name and the doctor's notes. The main point in this mapping is the declaration of the `doctor_notes` field as a `flattened` type.

Listing 4.32 Creating a mapping with the `flattened` data type

```
PUT consultations
{
  "mappings": {
    "properties": {
      "patient_name":{
        "type": "text"
      },
      "doctor_notes":{
        "type": "flattened"
      }
    }
  }
}
```

This field can consist of any number of subfields.

The field is declared as flattened.

The idea is that any field (and its subfields) declared `flattened` will not be analyzed; all the values are indexed as `keyword`s. Let's create a patient consultation document and index it.